BUILDING | 6 MIN READ

20 security holes in your vibe-coded app

The 20 holes that show up in almost every vibe-coded app, what each one costs you, and the exact prompt to paste into Claude or Cursor to audit your own codebase.

Vibe coding gets you a working app in a weekend. It also ships the same security holes over and over, because the model optimizes for working, not safe. Here are the 20 that show up constantly. Copy this whole page, paste it into Claude or Cursor with the prompt at the bottom, and fix what it finds.

1. Committing your .env to GitHub

Secrets in a public repo get scraped by bots within minutes of the push, and deleting the file later doesn't help because git keeps the history. The fix: add .env to .gitignore, keep secrets server-side, and rotate

every key that ever touched a commit.

2. Putting real API keys in your frontend code

Anything in your JS bundle ships to the browser. Prefixes like NEXT_PUBLIC_ and EXPO_PUBLIC_ mean public, and anyone can read them in devtools. The fix: only publishable keys go client-side. Real keys live in edge functions or your backend.

3. Leaving row level security off

On Supabase or Firebase with RLS disabled, your anon key is a master key. Anyone who pulls it out of your bundle can read and write every user's rows. The fix: RLS on by default, policies scoped to auth.uid(), and test them with a second account.

4. Checking permissions in the frontend

if (user.isAdmin) in React stops nobody. A user edits the response, calls your API directly, or just flips a variable in devtools. The fix: every permission check runs server-side. The frontend only decides what to show, never what's allowed.

5. No rate limiting on your endpoints

One script hammering your login brute-forces passwords. One script hammering your AI route runs your bill into four figures overnight. The fix: per-user and per-IP throttles on every login, signup, and expensive

route.

6. Building SQL queries with string concatenation

Splicing user input into a query string lets someone close your quote and append their own SQL. The fix: parameterized queries or an ORM. Never build a query with string interpolation.

7. No server-side input validation

Form validation is a UX feature. Anyone can skip your form and POST directly to the endpoint with a negative quantity or a 10MB "email." The fix: validate and sanitize on the server too, with a schema library like Zod.

8. Rendering user content as raw HTML

dangerouslySetInnerHTML on user-submitted text lets someone inject a script that runs in other users' browsers and steals their sessions. The fix: render user content as text. If you must render HTML, sanitize it with DOMPurify first.

9. Storing passwords in plaintext

One database leak exposes every account on your app, plus every other site where those users reused the password. The fix: use your auth provider's hashing, bcrypt or argon2. Don't write your own.

10. Keeping auth tokens in localStorage

Any XSS script on your page can read localStorage and walk off with a live session token. The fix: httpOnly cookies on web, secure storage like expo-secure-store on native.

11. An admin panel with no auth on it

/admin protected by nothing but the fact that you haven't linked to it. Same for the /debug route that dumps your environment variables. The fix: auth-gate every internal route and strip debug endpoints out of production builds.

12. CORS set to star

Access-Control-Allow-Origin: * lets any site on the internet call your API with a logged-in user's credentials attached. The fix: allowlist your own origins. Nothing else.

13. No email verification on signup

Unverified signups mean fake accounts, spam, and takeover flows that let someone claim an address they don't own. The fix: verify the email before granting access to anything real.

14. Predictable IDs with no ownership check

Sequential IDs are fine. Sequential IDs with no check that the record belongs to the requester means someone changes /order/1001 to /order/1002 and reads a stranger's data. The fix: on every fetch and

every mutation, confirm the resource belongs to the user asking for it.

15. Saving the whole request body on updates

An update endpoint that writes whatever JSON it receives lets a user add "role": "admin" or "is_premium": true to their own profile update. The fix: allowlist the exact fields a user can set and ignore everything else in the body.

16. Webhooks with no signature check

Your Stripe handler trusts any POST that looks right. Someone sends a fake payment_succeeded and unlocks premium for free. The fix: verify the signature against your webhook secret before you touch the payload.

17. Stack traces in production

Verbose errors leak file paths, table names, and library versions, which is a free map for anyone probing your app. The fix: generic error messages to users. Real details go to your logs.

18. Never updating your dependencies

Packages you installed a year ago have public CVEs with exploit code already written. The fix: run npm audit, turn on Dependabot, and patch on a schedule instead of when something breaks.

19. No password strength or breach check

Without a minimum length and a breached-password check, password123 becomes a live account on your app. The fix: set a minimum length and enable the leaked-password check your auth provider already ships. Supabase has one built in.

20. File uploads with no validation

Accepting any file lets someone upload a script and find a way to get your server to serve or execute it. The fix: check type and size, store uploads outside the web root or in object storage, and never execute an uploaded file.

The prompt to paste with it

Copy everything above, put this on top, and send the whole thing to Claude or Cursor:

```
Audit my codebase against the 20 issues below. For each one, tell me whethe
```

Save this one. Run the audit before you ship, and again after every big feature. Twenty minutes now beats a breach email later.

Murphmaxxing

You can build it. Start this weekend.

Work with me →

© 2026 Murphmaxxing. All rights reserved.

BUILT TO MAKE YOU SHARPER